

Attachment

Dear Reviewers,

In this attachment, we present complementary details to support our response in the rebuttal letter

Part 1. Cases of semantic changes

Case1: During the update of the following contracts, BCUpdater did not fully adhere to the instructions. The LLM fixed compilation errors caused by breaking changes but then omitted all other contract code in the output, resulting in missing functions in the updated contracts. Such semantic deviations are relatively easy for developers to identify.

```
constructor(string memory _name, string memory _symbol, string memory _uri, address _admin)
ERC721(_name, _symbol)
+ Ownable(msg.sender) // Added msg.sender as the initialOwner for Ownable
{
  _controller = _admin;
  __baseURI = _uri;
  _mint(msg.sender, ++totalSupply);
}
+ // ... (rest of the contract code remains unchanged)
- modifier onlyController {
- require(msg.sender == _controller || msg.sender == owner(), -"ONLY_CONTROLLER");
- _;
- }
- function verifyCode(bytes32 codeHash, bytes calldata signature) internal -view returns (bool) {
- bytes32 hash = keccak256(abi.encodePacked(address(this), msg.sender, -codeHash));
- return _signer == ECDSA.recover(hash, signature);
- }
...
```

Case2: During the update of the following contracts, BCUpdater removed functionalities related to PaymentSplitter due to the deprecation of the PaymentSplitter.sol library in OpenZeppelin. Developer confirmation is required to determine whether this functionality should be removed.

```

- import "@openzeppelin/contracts/finance/PaymentSplitter.sol";
constructor(
uint256 saleStart_,
string memory baseURIValue_,
address[] memory payees,
uint256[] memory paymentShares,
address reservedTokensAddress
) ERC721("Afro Droids", "AD")
+ Ownable(initialOwner) {
- PaymentSplitter(payees, paymentShares) {
    _baseURIValue = baseURIValue_;
    _saleStart = saleStart_;
    _presaleStart = _saleStart - (60 * 60 * 6);
    _reservedTokensAddress = reservedTokensAddress;
    _canAddToPresale[_reservedTokensAddress] = true;
}
...

```

Part 2: Update Log for Developer Review

Below is a developer-friendly analysis of the BCUpdater process and change points.

Analysis of Error Causes

The errors occurred because the Pausable and ReentrancyGuard contracts were moved to the utils directory in OpenZeppelin's library starting from version 5.0.0. Therefore, the import paths for these contracts need to be updated to reflect their new locations.

Main modifications

Updated the import statements for Pausable and ReentrancyGuard to their new locations in the utils directory:

```

Changed      import      "@openzeppelin/contracts/security/Pausable.sol";      to      import
"@openzeppelin/contracts/utils/Pausable.sol";
Changed      import      "@openzeppelin/contracts/security/ReentrancyGuard.sol";      to      import
"@openzeppelin/contracts/utils/ReentrancyGuard.sol";

```

Part 3: Handling of Contracts Exceeding Token Limit

The error location is identified using the output of the SOLC compiler. The corresponding contract is then extracted via the AST, and after being repaired by BCUpdater, it is replaced in the original contract.

1. Compiler output

```
Error: Expected string end-quote.  
  
--> text.sol:39:34:  
  
|  
339 | string public _metadataURI = "https:  
| ^^^^^^^
```

2. The location is identified based on the compiler output, and the contract's start and end positions are determined using the AST. The extracted contract is then sent to BCUpdater for repair.

```
# Contract to be repaired  
  
# --- Line 305 ---  
  
{Contract containing the error lines}  
  
# ---Line 429 ---
```

3. After updating the contract, BCUpdater replaces it at its original position in the source code.

```
# Contract to be repaired  
  
# --- Line 305 ---  
  
{Replace the contract at this location with the version updated by BCUpdater.}  
  
# ---Line 429 ---
```

Part 4: Quantitative Analysis of Code Changes

We conducted a quantitative analysis of the number of lines modified during contract updates based on the sampling strategy in RQ2, and detailed data has been made available in our open-source repository (Case Study/Change Analysis No-Lib.xlsx & Case Study/Change Analysis Lib.xlsx).

Table 1 below shows the average number of changes lines for the two datasets. It can be observed that the No-Lib-Contract has a higher average (14.6) compared to the Lib-Contract (5.6). This is because the No-Lib-Contract was constructed using

stratified sampling across Solidity versions, and older versions tend to require more syntax-related modifications.

An interesting observation is that although the Lib-Contract involves fewer line modifications, it was perceived as more difficult in the User Study. This is because updating contracts that involve third-party libraries imposes higher demands on the participants.

Table 1: Average Change Line Number of No-Lib-Contract and Lib-Contract

	No-Lib-Contract	Lib-Contract
Change Line Num	14.6	5.6

Table 2 shows the number of changed lines corresponding to different Solidity versions in the No-Lib-Contract. It can be observed that the higher the Solidity version, the more lines need to be changed.

Table 2: Average Change Line Number per Solidity Compiler Version

Solidity Version	0.4	0.5	0.6	0.7
Change Line Num	29.3	9.9	4.5	1.9